

TPE 4

ALGORITHME DE DIJKSTRA

Plus Court Chemin avec Poids

VOTRE MISSION

Vous travaillez pour ZEMIGO, une application de moto-taxi à Cotonou. Votre mission : implémenter l'algorithme de Dijkstra pour calculer les itinéraires les plus rapides entre les quartiers de la ville.

Ce TPE vous permettra de comprendre comment fonctionnent les GPS, les applications de navigation, et le routage Internet !

Repository GitHub : [graph-theory-course/session4/tpe/](https://github.com/graph-theory-course/session4/tpe/)

ARBORESCENCE DU PROJET

```
TPE4_Dijkstra/  
include/ Fichiers d'en-tête  
graph.h Classe Graph (fourni)  
dijkstra.h Déclarations des 5 fonctions  
src/ Code source  
graph.cpp Implémentation Graph (fourni)  
dijkstra.cpp À COMPLÉTER (vos TODO)  
main.cpp Programme principal  
data/ Graphes de test  
graph_test.txt Petit graphe pour debug  
graph_cotonou.txt Graphe réaliste Cotonou  
tests/ Tests automatisés  
test_dijkstra.cpp 18 tests  
Makefile Compilation automatique
```

LES 5 FONCTIONS À IMPLÉMENTER

Niveau	Fonction	Points	Objectif
BASE	dijkstra_base()	4	Version simple (recherche linéaire)
BASE	dijkstra_heap()	6	Version optimale (priority_queue)
STANDARD	reconstruct_path()	6	Reconstruction du chemin
STANDARD	dijkstra_between_two()	6	Distance entre 2 sommets

BONUS	compare_implementations()	3	Comparaison performance
CODE	Qualité code + commentaires	+2	Noms variables, explications
	TOTAL	27	Ramené sur 25 points

DEADLINE : Début de la Séance 5 (Arbre Couvrant Minimal)

COMPRENDRE DIJKSTRA AVANT DE CODER

L'IDÉE CENTRALE

Dijkstra trouve le plus court chemin en testant intelligemment les chemins possibles.

Comment ? En visitant TOUJOURS le sommet le plus proche parmi ceux pas encore visités.

C'est un algorithme GLOUTON (greedy) : à chaque étape, on fait le choix localement optimal, et ce choix est globalement optimal !

LA RELAXATION (Cœur de l'algorithme)

Pour chaque sommet u visité :

Pour chaque voisin v de u :

Si passer par u améliore la distance vers v :

Mettre à jour $distance[v]$

```
if (distance[u] + poids(u,v) < distance[v]) distance[v] = distance[u] + poids(u,v) parent[v] = u
```

POURQUOI ÇA MARCHE ?

INVARIANT : Une fois un sommet extrait (visité), sa distance est FINALE et OPTIMALE.

Pourquoi ? Car avec $poids \geq 0$, aucun chemin futur ne peut améliorer un sommet déjà visité.

C'est un algorithme GLOUTON (greedy) optimal !

DEUX VERSIONS

Version	Structure	Complexité	Usage
dijkstra_base()	Recherche linéaire	$O(V^2)$	Simple à comprendre
dijkstra_heap()	File de priorité (tas)	$O((V+E) \log V)$	Version professionnelle

`dijkstra_heap()` peut être jusqu'à 100× plus rapide !

PIÈGES À ÉVITER ABSOLUMENT

1. Oublier $distance[source] = 0 \rightarrow$ Tout reste inaccessible
2. Ne pas marquer `visited[]` $\rightarrow$ Boucle infinie
3. Ordre pair `priority_queue` : mettre $\{distance, vertex\}$ (pas l'inverse !)
4. Oublier `greater<>` $\rightarrow$ Max-Heap au lieu de Min-Heap
5. Reconstruire le chemin sans `reverse()` $\rightarrow$ Chemin à l'envers
6. Ne pas vérifier `visited[u]` après extraction dans `dijkstra_heap()`

GUIDE D'IMPLÉMENTATION

FONCTION 1 : dijkstra_base() [BASE - 4 points]

Objectif : Comprendre l'algorithme complet avec recherche linéaire du minimum

Complexité : $O(V^2)$

TODO : 6 étapes algorithmiques

```
vector dijkstra_base(Graph& g, int source) { // TODO 1 : Initialisation // distance[],
visited[], V, distance[source] = 0 // TODO 2 : Boucle principale (V itérations) for (int i = 0;
i < V; i++) { // TODO 3 : Trouver sommet u non visité avec distance min // TODO 4 : Marquer u
comme visité // TODO 5 : Relaxation - pour chaque voisin v de u } // TODO 6 : Retourner
distance[] }
```

FONCTION 2 : dijkstra_heap() [BASE - 6 points]

Objectif : Version optimale avec priority_queue (C'EST LE VRAI DIJKSTRA !)

Complexité : $O((V+E) \log V)$

ATTENTION : priority_queue est Max-Heap par défaut → utiliser greater<> !

```
vector dijkstra_heap(Graph& g, int source) { // TODO 1 : Initialisation (comme dijkstra_base) //
TODO 2 : Créer file de priorité (MIN-HEAP) priority_queue, vector>, greater>> Q; // TODO 3 :
Ajouter point de départ // TODO 4 : Boucle while (!Q.empty()) // TODO 5 : Extraire minimum +
vérifier visited[] // TODO 6 : Relaxation + ajout dans Q }
```

FONCTION 3 : reconstruct_path() [STANDARD - 6 points]

Objectif : Reconstruire le chemin avec parent[]

ATTENTION : Le chemin est à l'envers → utiliser reverse() !

```
vector reconstruct_path(vector& parent, int source, int dest) { // TODO 1 : Vérifier si dest
accessible (parent[dest] != -1) // TODO 2 : Cas spécial source == dest // TODO 3 : Remonter
parent[] de dest vers source // TODO 4 : Inverser le chemin avec reverse() }
```

PIÈGES FRÉQUENTS ET PRIORITY_QUEUE

PIÈGE #1 : Tout est inaccessible

Symptôme : Toutes les distances valent ∞

Cause : Vous avez oublié `distance[source] = 0`

Solution :

```
for (int i = 0; i < V; i++) distance[i] = INT_MAX; distance[source] = 0; // ← NE PAS OUBLIER !!!
```

PIÈGE #2 : Boucle infinie ou segfault

Symptôme : Le programme ne se termine jamais ou crash

Cause : Vous avez oublié de marquer `visited[]`

Solution :

```
// Juste après avoir trouvé le sommet u visited[u] = true; // ← TRÈS IMPORTANT !
```

PIÈGE #3 : Priority queue en désordre

Symptôme : `dijkstra_heap()` donne des résultats incorrects

Cause : Vous utilisez `priority_queue` sans `greater<>`

Solution :

```
// ■ FAUX (Max-Heap par défaut) priority_queue< Q; // ■ CORRECT (Min-Heap avec greater<>)
priority_queue< vector<, greater>> Q;
```

PIÈGE #4 : Ordre de la paire

Symptôme : Performances catastrophiques

Cause : Vous avez mis `{sommet, distance}` au lieu de `{distance, sommet}`

Solution :

```
// ■ FAUX Q.push({sommet, distance}); // ■ CORRECT Q.push({distance, sommet}); // La queue trie
sur le PREMIER élément !
```

PIÈGE #5 : Chemin à l'envers

Symptôme : Le chemin va de dest vers source au lieu de source vers dest

Cause : `parent[]` remonte de destination vers source

Solution :

```
// Après avoir reconstruit le chemin reverse(chemin.begin(), chemin.end());
```

PIÈGE #6 : Sommets traités plusieurs fois

Symptôme : `dijkstra_heap()` est très lent

Cause : Vous ne vérifiez pas `visited[]` après extraction

Solution :

```
while (!Q.empty()) { int u = Q.top().second; Q.pop(); if (visited[u]) continue; // ← ESSENTIEL  
! visited[u] = true; // ... relaxation }
```

TESTS, BARÈME ET RENDU

GRAPHES DE TEST

1. graph_test.txt (6 sommets, 9 arêtes)
→ Petit graphe simple pour debugger facilement
→ Tous les tests utilisent ce graphe
2. graph_cotonou.txt (8 quartiers, 14 routes)
→ Graphe réaliste : réseau routier simplifié de Cotonou
→ Optionnel pour fonction bonus

COMPILATION ET TESTS

Compiler : make

Exécuter : ./bin/main

Tests : make test && ./bin/test_dijkstra

Sortie attendue : 18 tests réussis sur 18

BARÈME DÉTAILLÉ

Fonction	Tests	Points	Critères clés
dijkstra_base()	4	4	Init, boucle V fois, trouver min, visited[], relaxation
dijkstra_heap()	5	6	priority_queue, greater<>, pair{dist,v}, visited[]
reconstruct_path()	4	6	Remonter parent[], reverse(), cas limites
dijkstra_between_two()	3	6	Arrêt anticipé if (u==dest), gestion inaccessible
compare_implementations()	2	3	Mesure temps, affichage comparatif, speedup
Qualité code	—	+2	Noms variables, commentaires, indentation
TOTAL	18	27	Ramené sur 25 points

FORMAT DE RENDU

Fichier à soumettre : NOM_Prenom_TPE4.zip

Contenu du ZIP :

- src/dijkstra.cpp (votre implémentation complète)
- capture_tests.png (screenshot des 18 tests réussis)

NE PAS INCLURE : fichiers .o, bin/, obj/, __pycache__/

CHECKLIST AVANT DE RENDRE

- Les 5 fonctions sont implémentées
- make compile sans erreurs ni warnings
- ./bin/test_dijkstra affiche 18/18 tests réussis
- Le code est commenté et bien indenté
- Les noms de variables sont explicites
- La complexité de chaque fonction est documentée
- Le fichier ZIP contient dijkstra.cpp + capture
- Le nom du ZIP respecte le format NOM_Prenom_TPE4.zip

CONSEILS POUR RÉUSSIR

1. Commencez par lire la page 2 (Comprendre Dijkstra)
2. Implémentez dans l'ordre : dijkstra_base → dijkstra_heap → reconstruct_path
3. Testez après CHAQUE fonction (ne pas tout faire d'un coup)
4. Utilisez graph_test.txt pour debugger (petit graphe = facile à tracer)
5. Relisez les pièges de la page 4 avant de coder dijkstra_heap
6. Si un test échoue, affichez les valeurs pour comprendre
7. Comparez vos résultats avec l'exécution manuelle sur papier
8. N'hésitez pas à demander de l'aide si vous êtes bloqué !

Bon courage !

Dijkstra est un des algorithmes les plus utilisés au monde (GPS, Internet, jeux vidéo...).

Le maîtriser est un atout majeur pour votre carrière !